

Paradigmi di programmazione

Contents

1	Introduzione	3
2	Descrizione dei paradigmi	4
2.1	Paradigma imperativo	4
2.2	Paradigma object-oriented	4
2.3	Paradigma funzionale	4
2.4	Programmazione concorrente	4
2.5	Programmazione parallela	4
3	Il ruolo chiave dei tipi	5
4	Linguaggi	6
4.1	Fortran	6
4.2	ALGOL 60	6
4.3	LISP	6
4.4	SIMULA 67	6
4.5	C	7
4.6	Pascal	7
4.7	Smalltalk	7
4.8	PROLOG	7
4.9	C++	7
4.10	Java	7
4.11	Altri linguaggi popolari	8
4.12	Come crescono i linguaggi?	8
5	Approccio scientifico	9
6	Lambda Calculus	10
6.1	Definizione	10
6.2	Lambda espressioni	10
6.2.1	Segnatura e funzione identità	10
6.2.2	Applicazione	10
6.2.3	Funzione costante	10
6.2.4	Funzioni annidate	11
6.2.5	Ordine di semplificazione	11
6.2.6	Funzioni come parametro	11
6.2.7	Prolungamento di espressione	11
6.2.8	Parametri attuali numerici	12
6.2.9	Ambiguità	12
6.3	Convenzioni sintattiche	12
6.4	Variabili libere e legate	12
6.5	Definizione sintattica	13
6.6	Alberi per lambda espressioni	13
6.7	Alpha equivalenza	13
6.8	Valutazione di lambda espressioni	13

1 Introduzione

Email Docente: chiara.bodei@unipi.it

E-Learning: <https://elearning.di.unipi.it/course/view.php?id=1167> Il Teams è privato, non si può entrare. (Future notice)

Lo scopo del corso è di facilitare l'apprendimento di nuovi linguaggi di programmazione attraverso lo studio dei paradigmi.

2 Descrizione dei paradigmi

Non esiste un paradigma migliore degli altri: la scelta dipende dalla situazione e dal problema da risolvere.

Molti programmi possono utilizzare, per sottoproblemi diversi, approcci differenti (**multiparadigma**).

2.1 Paradigma imperativo

Il **paradigma imperativo** descrive il programma come una sequenza di istruzioni che modificano lo stato del programma.

Il programmatore specifica **come** ottenere il risultato, indicando esplicitamente le operazioni da eseguire.

Esempi di costrutti tipici sono assegnamenti, cicli e istruzioni condizionali.

2.2 Paradigma object-oriented

Il **paradigma object-oriented** (o **a oggetti**) organizza il programma attorno a oggetti che combinano **dati** e **comportamenti**.

Gli oggetti appartengono a **classi**, che ne definiscono caratteristiche e operazioni.

I principali concetti sono **incapsulamento**, **ereditarietà**, **polimorfismo** e **astrazione**.

2.3 Paradigma funzionale

Il **paradigma funzionale** considera il programma come una composizione di **funzioni**, cercando di evitare modifiche dello stato e effetti collaterali.

Le funzioni possono essere trattate come valori, passate come argomenti e restituite come risultati.

Sono importanti concetti come **immutabilità**, **funzioni pure**, **ricorsione** e **funzioni di ordine superiore**.

2.4 Programmazione concorrente

La **programmazione concorrente** permette a più attività di essere **in esecuzione nello stesso intervallo di tempo**, anche se non vengono necessariamente eseguite simultaneamente.

Le attività possono alternarsi e comunicare o sincronizzarsi tra loro.

È utile, ad esempio, per gestire contemporaneamente richieste di utenti, operazioni di I/O o attività indipendenti.

2.5 Programmazione parallela

La **programmazione parallela** consiste nell'eseguire più operazioni **simultaneamente**, generalmente sfruttando più processori o core.

L'obiettivo principale è ridurre il tempo necessario per completare un'elaborazione, dividendo il problema in parti che possono essere eseguite in parallelo.

La programmazione parallela è quindi particolarmente utile per problemi scomponibili in attività indipendenti o parzialmente indipendenti.

3 Il ruolo chiave dei tipi

I **tipi** nella programmazione sono molto importanti, in quanto dichiarano gli intenti di dati e funzioni.

Il **Type Checking** (presente per esempio in TypeScript) assicura il corretto svolgimento delle operazioni.

Inoltre, il type checking rende il programma più efficiente ed evita errori nelle operazioni. Si dice che il type checking offra una **correttezza parziale** (solo per alcuni tipi di errori) ed è un esempio di tecnica di analisi di programma (statica e dinamica).

4 Linguaggi

4.1 Fortran

Fortran (FORMula TRANsfer) è uno dei linguaggi più antichi, realizzato da **John Backus** (1924-2007) negli anni 50 all'IBM, primo linguaggio di alto livello implementato efficacemente (memoria statica, senza annidamento).

Ha introdotto le variabili come le conosciamo oggi, i cicli, le procedure, le etichette delle istruzioni e molto altro.

Le prime versioni non supportavano la ricorsione e includevano caratteristiche oggi abbandonate.

4.2 ALGOL 60

ALGOL (ALGOritmic Language) fu progettato nel 1958-1960 come linguaggio universale.

Ha introdotto innovazioni fondamentali:

- Sintassi formalmente specificata (BNF)
- Scoping lessicale / blocchi
- Procedure modulari
- Passaggi per nome
- Procedure ricorsive
- Dichiarazioni di tipi di variabili
- Allocazione dello stack e record di attivazione

4.3 LISP

Il **LISP** (LISt Processor) è un linguaggio funzionale, sviluppato da **John McCarty** nel 1958 al MIT per la manipolazione di liste.

Precursore di molti concetti, tra cui la gestione dinamica della memoria (heap e garbage collector).

4.4 SIMULA 67

SIMULA 67 è stato sviluppato da **Ole-Johan** e **Kristen Nygaard**, è un'estensione di ALGOL 60.

Fu il primo linguaggio orientato agli oggetti, con classi, oggetti, ereditarietà e sottoclassi. Introdusse anche passaggi per valore e riferimento, puntatori e **co-routine** (simili ai thread).

4.5 C

Introdotta nei Bell Labs 1972 da **Ritchie**, il suo sviluppo è strettamente legato a **UNIX** e ai sistemi operativi.

Maggiori possibilità di accessi alle funzionalità di basso livello della macchina.

Gestione manuale della memoria (puntatori ma no garbage collection). Le funzioni non si possono annidare.

Controllo dei tipi non forte.

Linguaggio imperativo.

4.6 Pascal

progettato da **N. Wirth** agli inizi degli anni '70, prende il nome dal filosofo **Blaise Pascal**, inventore della **pascalina**, strumento di calcolo precursore della calcolatrice moderna. Rappresenta una versione più semplice di ALGOL 60.

Linguaggio fortemente tipizzato, gestione dinamica della memoria e scoping statico.

Offre molti tipi di dati (anche custom) come array, record, file e set.

Supporta le funzioni.

4.7 Smalltalk

Introdotta negli anni '70 da **Alan Kay** allo Xerox Palo Alto Research Center per supportare nozioni di incapsulamento e information hiding.

Tutto è un oggetto tranne le variabili; la computazione avviene tramite uno scambio di messaggi.

Utilizza il garbage collector ed è dotato di GUI e IDE integrati.

4.8 PROLOG

PROLOG (PROgrammation en LOGique) fu introdotta da **Robert Kowalski** (teoria), **Marten Van Emden** (dimostrazione sperimentale) e implementata finalmente da **Alain Colmerauer** all'inizio degli anni '70.

Linguaggio dichiarativo

4.9 C++

Introdotta nei Bell Labs nel 1986 da **Bjarne Stroustrup**.

Gli oggetti sono una generalizzazione di "struct".

Estende **C** con le classi ed un migliore sistema di tipi.

Supporta concetti complessi come l'ereditarietà multipla, modelli / generici (template) e la gestione delle eccezioni.

4.10 Java

Introdotta alla SUN 1991-1995 dal gruppo di **James Gosling**, completamente orientata agli oggetti.

Ha garbage collection e quindi la gestione della memoria non è esplicita; non c'è ereditarietà multipla e neanche generics.

Si possono usare thread in maniera concorrente.

Presta molto attenzione alla portabilità dato il suo aspetto ibrido tra linguaggio compilato e interpretato.

4.11 Altri linguaggi popolari

- **Imperativi:** Modula, Oberon, Ada
- **Funzionali:** Miranda, Haskell, ML, Caml, OCaml, Scheme, Common LISP
- **Orientati agli oggetti:** Objective-C, Eiffel, Modula-3, Self, C#, CLOS
- **Logici:** Gödel, LDL, ACL2, Isabelle, HOL
- **Scripting:** (Text processing) Perl, Python; (Web programming) JavaScript, PHP
- **Elaborazione dati e database:** Cobol, SQL, 4GL, XQuery
- **Programmazione di sistemi:** PL/I, PL/M, BLISS
- **Applicazioni specializzate:** APL, Forth, Icon, Logo, SNOBOL4, GPSS, Visual Basic
- **Concorrenti:** Concurrent Pascal, Concurrent C, C*, SR, Occam, Erlang, Obliq

4.12 Come crescono i linguaggi?

Ci sono diversi fattori che fanno crescere un linguaggio, eccone diversi:

- **Killer App:** un linguaggio è la scelta gettonata per sviluppare app di successo (Java, PHP, R)
- **Esclusività delle piattaforme:** alcuni linguaggi si impongono perchè vincolati a un ecosistema (Swift e iOS).
- **Velocità dell'upgrade:** evoluzione rapida che mantiene la semplicità (Python)
- **Grande slogan:** fa la differenza (Java: write once, run everywhere)
- **Slow and steady:** crescita lenta ma stabile e duratura (C)
- **Sintassi chiara:** abbassa la barriera d'ingresso (Python)
- **Job market:** orienta le scelte (Java, JavaScript, Python)
- **Community:** il contributo degli appassionati tramite numerosissime librerie (JavaScript, Python)

5 Approccio scientifico

Per studiare i linguaggi di programmazione si attuerà un **metodo scientifico**:

1. **Fatti:** la realtà che va studiata (osservazioni, esperimenti, dati...)
2. **Modello:** la rappresentazione tramite leggi matematiche.
3. **Ragionamento:** analisi della realtà attraverso il modello.

Questo metodo viene applicato in tutti i campi della scienza e ricerca e lo applicheremo anche noi in questo modo:

1. **Fatti:** linguaggi di programmazione.
2. **Modello:** semantica.
3. **Ragionamento:** comprensione e studio di proprietà.

6 Lambda Calculus

6.1 Definizione

Il **lambda calcolo** è stato introdotto nel 1935 da **Church** e può essere considerato il primo linguaggio di programmazione funzionale e setta proprio una base per studiare tale paradigma.

6.2 Lambda espressioni

6.2.1 Segnatura e funzione identità

Il lambda calculus utilizza questa segnatura:

$$\lambda x.x$$

- λ = introduce la funzionale
- x = parametro formale (1); corpo (2) la cui valutazione applicata all'input dà il valore in output.
- $.$ = separa la variabile dal corpo

La legge di corrispondenza corrisponde a:

$$f(x) = x$$

Ovvero la funzione identità.

6.2.2 Applicazione

Per attualizzare questa segnatura si fa così e si chiama **applicazione**:

$$(\lambda x.x)z = z$$

Con "z" come **parametro attuale**.

6.2.3 Funzione costante

Vediamo adesso un esempio di **funzione costante**:

$$\lambda x.y$$

$$(\lambda x.y)z = y$$

L'output non dipende dall'input (parametro formale).

6.2.4 Funzioni annidate

Addirittura, esistono annidamenti di funzioni:

$$\lambda x. \lambda y. x$$

Questo aspetta due parametri in input e restituisce sempre il primo, corrisponde a:

$$f(x, y) = x$$

$$((\lambda x. \lambda y. x)z)w = (\lambda y. z)w = z$$

La prima funzione non ritorna un valore, bensì un'espressione (o una funzione) che può essere usata a sua volta.

La parte dell'espressione che si riduce si chiama **redex**, si effettuano le sostituzioni in ordine e si osserva come si evolve per determinare un risultato.

6.2.5 Ordine di semplificazione

Spesso si può scegliere un ordine di semplificazione:

- Partendo dalla funzione più esterna
- Partendo dalla funzione più interna

Vediamo un esempio:

$$(\lambda x. ((\lambda y. y)x))z$$

Risolvendo prima l'esterna:

$$(\lambda x. ((\lambda y. y)x))z = \lambda(y. y)z = z$$

Risolvendo prima l'interna:

$$(\lambda x. ((\lambda y. y)x))z = \lambda(x. x)z = z$$

6.2.6 Funzioni come parametro

Vediamo adesso un altro caso:

$$(\lambda f. fz)(\lambda x. x)$$

Questo usa un'intera funzione come parametro:

$$(\lambda f. fz)(\lambda x. x) = (\lambda x. x)z = z$$

6.2.7 Prolungamento di espressione

Le espressioni possono ricostruirsi:

$$(\lambda x. (xx))(\lambda y. y)$$

$$(\lambda x. (xx))(\lambda y. y) = (\lambda y. y)(\lambda y. y) = \lambda y. y$$

6.2.8 Parametri attuali numerici

Adesso vediamo un esempio con una somma:

$$((\lambda x. \lambda y. x + y)3)5$$

$$((\lambda x. \lambda y. x + y)3)5 = (\lambda y. 3 + y)5 = 3 + 5 = 8$$

Basta effettuare un passaggio alla volta.

6.2.9 Ambiguità

Vediamo una situazione di ambiguità con 3 espressioni concatenate nella forma (e_1, e_2, e_3) :

$$(\lambda x. \lambda y. xy)(\lambda x. x)z$$

Per convenzione bisogna svolgere le prime due e lasciare la terza per dopo:

$$(\lambda x. \lambda y. xy)(\lambda x. x)z = (\lambda y. ((\lambda x. x)y))z = \lambda(x.x)z = z$$

Questo conta anche per i parametri attuali:

$$(\lambda x. \lambda y. xy)zk$$

$$(\lambda x. \lambda y. xy)zk = (\lambda y. zy)k$$

Bisogna fare attenzione ad ambiguità tra parametri formali e attuali:

$$(\lambda x. \lambda y. xy)y$$

Se risolto, potrebbero esserci ambiguità, per questo conviene cambiare nome:

$$y = z$$

6.3 Convenzioni sintattiche

Per prima cosa, la **portata** (scope) del costruttore lambda si estende il più a destra possibile.

$$\lambda x. \lambda y. xy \text{ è uguale a } \lambda x. (\lambda y. (xy))$$

L'applicazione funzionale associa a **sinistra**:

$$e_1 e_2 e_3 \text{ diventa } (e_1 e_2) e_3$$

6.4 Variabili libere e legate

Una **variabile legate** (**bound**) a un λ sono quelle che fanno parte di una lambda espressione, ovvero una dichiarazione (parametri formali).

Le **variabili libere** (**unbound**) invece corrispondono al contrario, come i parametri attuali, le variabili non enunciate da un λ , queste sono definite dall'insieme:

$$FV$$

Per esempio, possiamo applicarlo ad un'espressione:

$$FV(x) = \{x\}$$

$$FV((\lambda x. x)z) = \{z\}$$

Un termine si dice **chiuso** se:

$$Fv(E) = \emptyset$$

6.5 Definizione sintattica

- $e ::= x$
 - $\lambda x.e$ (function declaration)
 - $e e$ (function call)

6.6 Alberi per lambda espressioni

Un altro tipo di rappresentazione sono gli alberi, fatti in questo modo:

- Il costruttore λ è la radice.
- Sul ramo sinistro si trova il parametro formale.
- Sul ramo destro si trova il corpo, il quale può ricominciare con un λ .

6.7 Alpha equivalenza

Due espressioni sono dette **alpha equivalenza** se differiscono solo per i nomi delle variabili legate. Più semplicemente, si potrebbe "rinominare" qualche parametro per rendere le due espressioni identiche, questa operazione si chiama **alpha-conversione**.

Questa tecnica può essere applicata anche a problemi di ambiguità legati alla similarità tra nomi di parametri formali e attuali.

Vediamo alcuni esempi:

Sostituzione negata, variabile legata

$$((\lambda x.yx)w)\{x := \lambda k.kx\} = (\lambda x.yx)w$$

Alpha equivalenza

$$((\lambda x.yx)w)\{y := \lambda k.kx\} \equiv_{\alpha} ((\lambda z.yz)w)\{y := \lambda k.kx\} = (\lambda z.\lambda k.kxz)w$$

Sostituzione semplice, diretta

$$((\lambda x.yx)w)\{w := \lambda k.kx\} = (\lambda x.yx)(\lambda k.kx)$$

6.8 Valutazione di lambda espressioni

La notazione è questa per la valutazione:

$$eval(espressione)$$

Semplicemente, la valutazione corrisponde a:

- Osservare se bisogna effettuare un'alpha-conversione per risolvere ambiguità.
- Sostituire i parametri formali coi parametri attuali.
- Svolgere la funzione (equivalente di effettuare una chiamata ad una funzione, per capire).

- Valutare il risultato e restituirlo.

La notazione per la sostituzione è questa:

$$e_1\{x := e_2\}$$

Nozione alternativa:

$$e_1\{e_2/x\}$$

La sostituzione può essere distribuita:

$$(e_1e_2)\{x := e\} \equiv (e_1\{x := e\})(e_2\{x := e\})$$

Altri concetti importanti:

$$(\lambda y.e_1)\{x := e\} \equiv \lambda y.(e_1\{x := e\}), y \neq x \wedge y \notin FV(e)$$

ATTENZIONE: se nell'espressione con cui si sta effettuando la sostituzione (non la variabile da sostituire) contiene variabili con lo stesso nome di variabili legate bisogna prima effettuare l'alpha conversione per evitare ambiguità ed errori. Infatti, la sostituzione si può effettuare solo sulle variabili non legate.